Documentation Complète

Race Simulation RL

Simulateur de course automobile avec Reinforcement Learning

Projet : race-simulation-rl (TUM — Heilmeier, Thomaser et al.)
Documentation générée : Novembre 2026

Documentation Complète — Race Simulation RL

Projet : `race-simulation-rl`
Auteur original : Alexander Heilmeier, André Thomaser, TUM
Objectif de cette doc : Comprendre la totalité du simulateur pour y intégrer du Reinforcement Learning
Date : Novembre 2026

Table des matières

Vue d'ensemble du projet

Architecture et arborescence

Workflow global

Module `racesim` — Le simulateur principal

- 4.1 `race.py` — Classe Race (cœur de la simulation)
- 4.2 `driver.py` — Classe Driver
- 4.3 `car.py`, `combustioncar.py`, `electriccar.py` — Classe Car
- 4.4 `tireset.py` — Classe Tireset
- 4.5 `track.py` — Classe Track
- 4.6 `race_reinftrain.py` — Sous-classe RL
- 4.7 `vse.py` — Virtual Strategy Engineer (VSE)
- 4.8 Les 4 variantes de VSE
- 4.9 `race_handle.py` — Wrapper de simulation
- 4.10 `import_pars.py` / `check_pars.py`

Module `racesim_basic` — Optimisation de stratégie

Module `helper_funcs` — Fonctions utilitaires

Module `machine_learning` — Préprocesseur

Module `machine_learning_rl_training` — Entraînement RL

- 8.1 `rl_environment_single_agent.py` — Environnement Gym-like
- 8.2 `rl_environment_multi_agent.py` — Environnement multi-agent
- 8.3 `rl_evaluate_policy.py` — Évaluation

Fichiers d'entrée — Paramètres `.ini`

Script principal `main_racesim.py`

Script principal `main_train_rl_agent_dqn.py`

Modèle physique — Calcul des temps au tour

Référence RL complète — Espace d'états, actions, récompenses

Guide pour modifier / étendre le RL

1. Vue d'ensemble du projet

Ce projet est un **simulateur de course de circuit au tour-à-tour** (lap-wise), conçu pour la F1 mais adaptable à d'autres séries. Son but principal est la **détermination de la stratégie de course optimale** (nombre d'arrêts au stand, tours d'entrée au stand, choix de pneumatiques, ravitaillement).

Le simulateur supporte :

- Simulation déterministe classique

- Simulation probabiliste (Monte Carlo)
- Entraînement d'un agent de Reinforcement Learning (RL) pour prendre des décisions de stratégie

[Diagramme Mermaid — voir la documentation Markdown]

■■ **Important :** Le projet contient **deux simulateurs** : `racesim_basic` (simple, sans adversaires, pour optimisation rapide) et `racesim` (complet, avec tous les pilotes, FCY, dépassements, RL).

---

2. Architecture et arborescence

```
race-simulation-rl/
■
■■■ main_racesim.py                    # Point d'entrée : simulation complète
■■■ main_racesim_basic.py              # Point d'entrée : simulation basique
■■■ main_train_rl_agent_dqn.py         # Point d'entrée : entraînement DQN
■■■ requirements.txt
■■■ setup.cfg
■
■■■ racesim/                           # Simulateur complet
■   ■■■ src/
■   ■   ■■■ race.py                     # ■ Classe principale Race
■   ■   ■■■ race_handle.py              # Wrapper instanciant Race
■   ■   ■■■ race_reinftrain.py          # Sous-classe Race pour RL
■   ■   ■■■ driver.py                   # Classe Driver
■   ■   ■■■ car.py                      # Classe Car (base)
■   ■   ■■■ combustioncar.py            # Voiture thermique
■   ■   ■■■ electriccar.py              # Voiture électrique
■   ■   ■■■ tireset.py                  # Classe Tireset (pneus)
■   ■   ■■■ track.py                    # Classe Track (circuit)
■   ■   ■■■ vse.py                      # ■ Virtual Strategy Engineer
■   ■   ■■■ vse_basestrategy.py         # VSE : stratégie de base
■   ■   ■■■ vse_realstrategy.py         # VSE : stratégie réelle
■   ■   ■■■ vse_supervised.py           # VSE : ANN supervisé
■   ■   ■■■ vse_reinforcement.py        # VSE : DQN entraîné (inférence)
■   ■   ■■■ import_pars.py              # Chargement des paramètres
■   ■   ■■■ check_pars.py               # Validation des paramètres
■   ■   ■■■ mcs_analysis.py             # Analyse Monte Carlo
■   ■   ■■■ _race_montecarlo.py         # Méthodes Monte Carlo (mixin)
■   ■   ■■■ _race_raceanalysis.py       # Méthodes d'analyse (mixin)
■   ■■■ input/
■       ■■■ parameters/                # Fichiers .ini par course (121 courses F1 2014-2019)
■       ■■■ vse/                       # Modèles pré-entraînés VSE
■
■■■ racesim_basic/                     # Simulateur simplifié
■   ■■■ src/
■       ■■■ calc_racetimes_basic.py    # Calcul temps de course simplifié
■       ■■■ opt_strategy_basic.py      # Optimisation de stratégie
■       ■■■ import_pars.py
■       ■■■ check_pars.py
■
■■■ machine_learning/
■   ■■■ src/
■       ■■■ preprocessor.py            # Préprocesseur features ML
■
■■■ machine_learning_rl_training/
■   ■■■ src/
■       ■■■ rl_environment_single_agent.py  # ■ Env RL mono-agent (tf-agents)
■       ■■■ rl_environment_multi_agent.py   # Env RL multi-agent
■       ■■■ rl_evaluate_policy.py           # Évaluation de politique RL
■
■■■ helper_funcs/
```

```
■■■ src/
    ■■■ calc_tire_degradation.py   # Modèle de dégradation des pneus
    ■■■ get_strat_combinations.py  # Combinaisons de stratégies
    ■■■ progressbar.py             # Barre de progression console
```

---

3. Workflow global

[Diagramme Mermaid — voir la documentation Markdown]

Étape 1 — Stratégie de base (`racesim_basic`)

Calcule la stratégie optimale (nombre d'arrêts, tour d'entrée, composé de pneus) **sans adversaires** pour minimiser le temps de course. Résultat : un fichier `.ini` avec les `strategy_info`.

Étape 2 — Entraînement RL (`main_train_rl_agent_dqn.py`)

Entraîne un agent DQN dans le simulateur complet pour **apprendre à prendre des décisions de stratégie**. L'environnement est un `PyEnvironment` de `tf-agents`. Sortie : un modèle `.tflite` et un préprocesseur `.pkl`.

Étape 3 — Simulation complète (`main_racesim.py`)

Lance la simulation pour 1 ou N courses. Peut utiliser le VSE de type `reinforcement` pour certains pilotes.

---

4. Module `racesim` — Le simulateur principal

4.1 `race.py` — Classe Race (cœur de la simulation)

Fichier : `race.py`

La classe `Race` hérite de `MonteCarlo` et `RaceAnalysis` (mixins). C'est le cœur de tout le système.

Attributs principaux (slots)

Attribut	Type	Description
---	---	---
`cur_lap`	`int`	Tour courant (variable de discrétisation)
`drivers_list`	`List[Driver]`	Liste des objets pilotes
`track`	`Track`	Objet circuit
`vse`	`VSE`	Virtual Strategy Engineer
`laptimes`	`ndarray [laps+1, drivers]`	Temps au tour de chaque pilote à chaque tour
`racetimes`	`ndarray [laps+1, drivers]`	Temps de course cumulé
`positions`	`ndarray [laps+1, drivers]`	Position à chaque tour
`bool_driving`	`ndarray [laps+1, drivers]`	`True` si le pilote roule (non retraité)
`fcy_data`	`dict`	Phases FCY (Full Course Yellow : SC ou VSC)
`retire_data`	`dict`	Données de retrait de course

`result_status`	`int`	Validité du résultat (`0` = valide)

Codes `result_status`

Code	Signification
---	---
`-1`	Résultat non disponible
`0`	■ Résultat valide
`1`	Tous les pilotes se sont retirés
`10`	Boucle infinie dans le dépassement
`11`	Temps au tour ≠ temps de course
`12`	Positions non plausibles
`13`	Distances minimales non respectées
`14`	Phases FCY présentes mais n'apparaissent pas
`15`	Pilote n'a pas utilisé 2 composés différents

Méthode `simulate_race()`

```
def simulate_race(self) -&gt; None:
    while self.cur_lap &lt; self.race_pars["tot_no_laps"]:
        self.__simulate_lap()
```

Boucle tour par tour. Chaque appel à `__simulate_lap()` exécute dans l'ordre :

[Diagramme Mermaid — voir la documentation Markdown]

Calcul du temps au tour `__calc_laptimes()`

Le temps au tour de base est :

$$t_{lap} = t_q + t_{gap_racepace} + \Delta t_{driver} + \Delta t_{car}$$

Avec les contributions additionnelles :

- **Dégradation des pneus** : `t_add_tireset()` → modèle polynomial sur l'âge
- **Perte de masse** (carburant) : `t_lap_sens_mass * m_fuel` (thermique uniquement)
- **Tour de départ** : `t_loss_firstlap + (p_grid - 1) * t_loss_pergridpos`
- **DRS** : `-t_drseffect / 2.0` si dans la fenêtre DRS (< 1 seconde)
- **FCY** : temps gonflé à ~140% (VSC) ou ~160% (SC) du temps normal

---

4.2 `driver.py` *— Classe Driver*

Fichier : `driver.py`

Représente un pilote avec ses paramètres et son historique de stratégie.

Attributs clés

Attribut	Type	Description
---	---	---

`initials`	`str`	Initiales, ex. `"VET"`
`team`	`str`	Équipe, ex. `"Ferrari"`
`t_driver`	`float`	Pénalité de temps due aux skills du pilote (s)
`t_lap_var_sigma`	`float`	Sigma de variation aléatoire du temps au tour
`strategy_info`	`List[List]`	■ Liste de stints : `[inlap, compound, age_start, refuel]`
`p_grid`	`int`	Position de départ sur la grille
`vel_max`	`float`	Vitesse maximale (km/h) pour dépassements
`p_accident`	`float`	Probabilité d'accident
`car`	`CombustionCar` ou `ElectricCar`	Objet voiture

`strategy_info` *— Format critique pour la stratégie*

```
# Exemple : départ en A3, arrêt au tour 25 pour A5, arrêt au tour 48 pour A2
strategy_info = [
    [0,  "A3", 0, 0.0],   # stint 0 : départ, composé A3, pneu neuf, pas de ravitaillement
    [25, "A5", 0, 0.0],   # stint 1 : inlap = tour 25, composé A5
    [48, "A2", 0, 0.0],   # stint 2 : inlap = tour 48, composé A2
]
```

C'est cette liste que le VSE modifie dynamiquement pour décider d'un arrêt.

Méthode `calc_basic_timeloss()`

```
def calc_basic_timeloss(self, use_prob_infl: bool, t_lap_sens_mass: float) -&gt; float:
    return self.car.t_add_car(t_lap_sens_mass=t_lap_sens_mass) + self.t_driver
    # + bruit gaussien si use_prob_infl=True
```

4.3 `car.py`, `combustioncar.py`, `electriccar.py` *— Classe Car*

Fichier : `car.py`

La classe `Car` est abstraite. `CombustionCar` et `ElectricCar` la spécialisent.

Attributs clés

Attribut	Description
---	---
`t_car`	Pénalité temps lié à la performance de la voiture
`drivetype`	`"combustion"` ou `"electric"`
`p_failure`	Probabilité de panne mécanique
`t_pit_tirechange_add`	Temps supplémentaire d'arrêt au stand (spécifique à l'équipe)
`m_fuel`	Masse de carburant courante (thermique)
`b_fuel_perlap`	Consommation par tour (thermique)

`energy`	Énergie courante (électrique)
`tireset`	Instance `Tireset` courante

Méthodes importantes

- `drive_lap()` : incrémente l'âge des pneus, décrément carburant/énergie
- `change_tires()` : crée un nouveau `Tireset` (lors d'un arrêt)
- `refuel()` : ajoute du carburant/de l'énergie
- `t_add_car()` : retourne la pénalité temps totale (pneus + performance voiture)
- `t_add_pit_standstill()` : retourne la durée de l'arrêt fixe au stand

4.4 `tireset.py` — Classe Tireset

Fichier : `tireset.py`

Composés disponibles

`'A1'`, `'A2'`, `'A3'`, `'A4'`, `'A5'`, `'A6'`, `'A7'` (sec, du plus dur au plus tendre), `'I'` (intermédiaire), `'W'` (pluie)

Attributs

Attribut	Description
---	---
`compound`	Composé actuel
`age_tot`	Âge total en tours
`age_curstint`	Âge dans le stint courant
`age_degr`	Âge "virtuel" pour le calcul de dégradation (tient compte des FCY)

`t_add_tireset()` — *Pénalité de dégradation*

Appelle `calc_tire_degradation()` avec l'âge courant, retourne la pénalité temps en secondes.

```
t_add_tireset = calc_tire_degradation(
    tire_age_start=self.age_degr,
    stint_length=1,
    compound=self.compound,
    tire_pars=self.tireset_pars
)
```

■ **Note :** Si `age_curstint == 0` (premier tour du stint), on ajoute `t_add_coldtires` pour simuler les pneus froids.

4.5 `track.py` — Classe Track

Fichier : `track.py`

Contient tous les paramètres physiques du circuit.

Paramètre	Unité	Description

| --- | --- | --- |
`t_q`	s	Meilleur temps de qualification
`t_gap_racepace`	s	Écart qualif → rythme de course
`t_lap_sens_mass`	s/kg	Sensibilité du temps au tour à la masse
`t_lap_fcy`	s	Temps estimé sous drapeau jaune FCY
`t_lap_sc`	s	Temps estimé derrière SC
`t_pit_tirechange_min`	s	Durée minimale d'arrêt au stand
`t_pitdrive_inlap`	s	Perte de temps en entrant dans les stands
`t_pitdrive_outlap`	s	Perte de temps en sortant des stands
`t_loss_pergridpos`	s/pos	Perte de temps par position de grille
`t_loss_firstlap`	s	Perte au départ (arrêt)
`t_gap_overtake`	s	Seuil de dépassement (track-dependent)
`t_drseffect`	s	Gain DRS (négatif)

4.6 `race_reinftrain.py` — Sous-classe RL

Fichier : `race_reinftrain.py`

`RaceReinftrain` hérite de `Race` et **redéfinit le découpage d'un tour** pour l'adapter au RL : chaque tour est découpé en deux phases pour permettre à l'agent de prendre sa décision au moment clé.

[Diagramme Mermaid — voir la documentation Markdown]

Méthodes clés :

- `__simulate_lap_start()` : Simule jusqu'à la décision VSE (état observable)
- `__simulate_lap()` : Applique la décision et finalise le tour
- `__simulate_lap_end()` : Finalise la dernière tour et effectue les vérifications
- `execute_presim_average_laptimes()` : Pré-simulation pour calculer les temps moyens de référence (utilisés dans la récompense)

4.7 `vse.py` — Virtual Strategy Engineer (VSE)

Fichier : `vse.py`

Le VSE est le **décideur de stratégie**. Il est appelé à chaque tour (`__handle_vse()`) et peut décider d'un arrêt au stand pour chaque pilote.

Méthode centrale `decide_pitstop()`

Reçoit l'état complet de la course et retourne une liste `next_compounds` :

- `None` → pas d'arrêt
- `"A3"` → arrêt et montage de composé A3

Inputs clés :

Paramètre	Description

| --- | --- |
`cur_compounds`	Composé actuel de chaque pilote
`tire_ages`	Âge des pneus de chaque pilote
`positions_prevlap`	Positions au tour précédent
`cur_lap` / `tot_no_laps`	Tour courant / total
`fcy_types` / `fcy_start_end_progs`	État des phases FCY
`used_2compounds`	A-t-on déjà utilisé 2 composés différents ?
`cur_positions`	Positions actuelles
`cur_racetimes_tmp`	Temps de course estimés (tour courant)

Features calculées en interne :

- `est_pos_losses` : nombre de positions perdues si arrêt maintenant
- `close_behinds` / `close_aheads` : pilote dans 1,5s derrière/devant
- `defendable_undercuts` : contre-stratégie défendable possible
- `fcy_stat_curlap` : état de la phase FCY (0-4)

4.8 Les 4 variantes de VSE

`basestrategy` — `vse_basestrategy.py`
Suit une stratégie pré-définie stockée dans le fichier `.ini`. Aucune décision adaptative.

`realstrategy` — `vse_realstrategy.py`
Rejoue exactement la stratégie réelle de la course historique.

`supervised` — `vse_supervised.py`
Deux réseaux de neurones entraînés sur données réelles F1 :

- **TC network** (*tire change*) : décide si on s'arrête
- **CC network** (*compound choice*) : choisit le composé

`reinforcement` — `vse_reinforcement.py`
Un seul réseau DQN (Q-network) entraîné dans le simulateur. Prédit les **Q-values** pour chaque action possible :

- Action 0 : pas d'arrêt
- Action 1-3 : arrêt avec composé A_x

Le modèle est chargé au format **TF Lite** (`.tflite`) pour l'inférence rapide.

Features d'entrée (preprocessées, normalisées) :

Feature	Description
`raceprogress_curlap`	Tour courant / total tours
`rel_position`	Position relative (0 = 1er, 1 = dernier)
`rel_est_pos_loss`	Perte de positions estimée si arrêt
`tireageprogress_corr`	Âge des pneus / total tours

`cur_compound`	Composé actuel (one-hot encodé)
`used_2compounds`	Bool : 2 composés déjà utilisés
`fcy_stat_curlap`	Statut FCY (0-4)
`close_behind`	Bool : adversaire à < 1,5s derrière
`close_ahead`	Bool : adversaire à < 1,5s devant
`defendable_undercut`	Bool : undercut défendable
`driver_initials`	Initiales pilote (one-hot encodé)

4.9 `race_handle.py` — Wrapper de simulation

Fichier : `race_handle.py`

Wrapper léger qui instancie `Race`, appelle `simulate_race()` et retourne les résultats. Utilisé par `main_racesim.py` pour les calculs parallèles (multiprocessing).

4.10 `import_pars.py` / `check_pars.py`

`import_pars.py` charge les fichiers `.ini` et retourne un dictionnaire `pars_in` contenant :

- `race_pars` : paramètres de course (tours totaux, participants, DRS, ...)
- `driver_pars` : paramètres par pilote
- `car_pars` : paramètres par voiture/équipe
- `tireset_pars` : paramètres de pneus par pilote
- `track_pars` : paramètres du circuit
- `vse_pars` : types VSE et stratégies
- `monte_carlo_pars` : paramètres pour la simulation probabiliste
- `event_pars` : phases FCY et retraits

---

5. Module `racesim_basic` — Optimisation de stratégie

Fichiers : `calc_racetimes_basic.py`, `opt_strategy_basic.py`

Simulation simplifiée (piste libre, un seul pilote) pour trouver rapidement la stratégie optimale. Utilisée aussi par `RaceReinftrain.execute_presim_average_laptimes()` pour calculer les temps de référence du reward RL.

Paramètres d'entrée similaires au simulateur complet mais sans interactions entre pilotes ni FCY.

---

6. Module `helper_funcs` — Fonctions utilitaires

`calc_tire_degradation.py`

Fichier : `calc_tire_degradation.py`

Calcule la pénalité de dégradation des pneus selon différents modèles :

Modèle	Formule

| --- | --- |
Linéaire (`lin`)	$t_{tire} = k_0 + k_1 \cdot age$
Quadratique (`quad`)	$t_{tire} = k_0 + k_1 \cdot age + k_2 \cdot age^2$
Cubique (`cub`)	$t_{tire} = k_0 + k_1 \cdot age + k_2 \cdot age^2 + k_3 \cdot age^3$
Logarithmique (`ln`)	$t_{tire} = k_0 + k_1 \cdot \ln(k_2 \cdot age + 1)$

Les coefficients (`k_0`, `k_1`, ...) sont définis par composé dans les fichiers `.ini`.

---

7. Module `machine_learning` — Préprocesseur

Fichier : `preprocessor.py`

Classe `Preprocessor` qui gère :

- **Encodage one-hot** des features catégorielles (composé, initiales du pilote, ...)
- Conversion chaîne → entier via `cat_dict`
- Méthodes `fit()`, `transform()`, `fit_cat_dict()`, `transform_cat_dict()`

Utilisé par l'environnement RL pour préparer les observations.

---

8. Module `machine_learning_rl_training` — Entraînement RL

8.1 `rl_environment_single_agent.py` — Environnement Gym-like

Fichier : `rl_environment_single_agent.py`

Classe `RaceSimulation` héritant de `tf_agents.environments.py_environment.PyEnvironment`.

Paramètres du constructeur

Paramètre	Défaut	Description
`race_pars_file`	—	Fichier de paramètres de course
`mcs_pars_file`	—	Fichier paramètres Monte Carlo
`vse_type`	`'supervised'`	VSE utilisé pour les autres pilotes
`use_prob_infl`	`True`	Active les influences probabilistes
`create_rand_events`	`True`	Crée aléatoirement SC/VSC et retraits

Espace d'actions (`action_spec`)

```
BoundedArraySpec(shape=(), dtype=np.int32, minimum=0, maximum=len(available_compounds))
```

- `0` → Pas d'arrêt
- `1` → Arrêt + composé `available_compounds[0]` (le plus dur)
- `2` → Arrêt + composé `available_compounds[1]`
- `3` → Arrêt + composé `available_compounds[2]` (le plus tendre)

Espace d'observation (`observation_spec`)

```
BoundedArraySpec(
    shape=(17 + len(available_compounds) + len(all_driver_initials),),
    dtype=np.float32, minimum=0.0, maximum=1.0
)
```

Composé de **4 features numériques** + **features catégorielles one-hot** :

Index	Feature	Normalisation
---	---	---
`[0]`	`cur_lap`	`/ tot_no_laps`
`[1]`	`position`	`(pos - 1) / (no_drivers - 1)`
`[2]`	`est_pos_loss`	`/ (no_drivers - 1)`
`[3]`	`tire_age`	`/ tot_no_laps`
`[4+]`	Features catégorielles (one-hot)	0 ou 1

Features catégorielles (one-hot encodées) :

- `cur_compound` (len = nb composés secs disponibles)
- `used_2compounds` (bool → 1/0)
- `fcy_stat_curlap` (5 catégories : 0-4)
- `close_behind` (bool)
- `close_ahead` (bool)
- `defendable_undercut` (bool)
- `driver_initials` (len = nb pilotes)

Récompense — Décomposition

La récompense est la somme de 3 composantes :

1. Reward Lap Time (`__calculate_reward_laptime`) :

$$r_{lap} = t_{avg_presim} - t_{cur_lap}$$

Positif si le pilote fait un tour plus rapide que son meilleur temps moyen théorique. Sous SC/VSC, le temps de référence est ajusté avec le temps FCY.

2. Reward Position (`__calculate_reward_position`) :

$$r_{pos} = 5 \times (\Delta_{position_gagnées})$$

+5 par position gagnée, -5 par position perdue depuis la dernière décision.

3. Reward Final Position (fin de course uniquement, `__calculate_reward_final_position`) :

$$r_{final} = position_theo - position_réelle$$

Compare la position finale réelle à la position théorique (basée sur les temps moyens de pré-simulation). Positif si l'agent a fait mieux que sa "juste" performance.

Pénalité : -10 si le pilote ne respecte pas la règle des 2 composés différents.

Reward total :

$$R_{total} = 0.1 \times (r_{lap} + r_{pos}) + r_{final}$$

Méthode `_reset()`

Crée une nouvelle course

Choisit **aléatoirement** un pilote parmi tous pour être contrôlé par l'agent
Simule le début du premier tour
Retourne la première observation

Méthode `_step(action)`

Applique l'action (ajoute un arrêt à `strategy_info` si action > 0)
Simule le tour suivant
Calcule l'observation et la récompense
Termine l'épisode si `cur_lap >= tot_no_laps`

8.2 `rl_environment_multi_agent.py` *— Environnement multi-agent*

Fichier : `rl_environment_multi_agent.py`

Variante où **tous les pilotes** sont contrôlés simultanément par l'agent. L'environnement est **batché** (un batch = un pilote par tour).

> ■■ **Avertissement :** L'évaluation post-entraînement (`calculate_final_positions`) n'est pas implémentée pour le multi-agent.

8.3 `rl_evaluate_policy.py` *— Évaluation*

Fichier : `rl_evaluate_policy.py`

Évalue un modèle TF Lite entraîné sur un grand nombre de courses (`num_races_postproc`) et affiche les distributions de positions finales.

---

9. Fichiers d'entrée — Paramètres `.ini`

Répertoire : `racesim/input/parameters/`

Chaque course a un fichier `.ini`, ex. `pars_Spielberg_2019.ini`. Structure principale :

```
[RACE_PARS]
race = Spielberg
season = 2019
tot_no_laps = 71
participants = VER, LEC, ...
use_drs = True
drs_window = 1.0
drs_allow_lap = 3
...

[TRACK_PARS]
name = Spielberg
t_q = 64.3
t_gap_racepace = 1.5
t_lap_sens_mass = 0.034
t_pit_tirechange_min = 2.5
t_pitdrive_inlap = 18.3
...

[DRIVER_PARS_VER]
carno = 33
```

```
name = Max Verstappen
team = RedBull
t_driver = 0.0
strategy_info = [[0, A3, 0, 0.0], [25, A5, 0, 0.0]]
p_grid = 1
...

[CAR_PARS_RedBull]
manufacturer = RedBull
t_car = 0.3
drivetype = combustion
t_pit_tirechange_add = 0.4
...

[TIRESET_PARS_VER]
tire_deg_model = lin
A3 = {k_0: 0.5, k_1_lin: 0.03}
A5 = {k_0: 0.3, k_1_lin: 0.05}
...

[VSE_PARS]
# Type de VSE par pilote : basestrategy / realstrategy / supervised / reinforcement
VER = reinforcement
LEC = supervised
...
base_strategy_VER = [[0, A3, 0, 0.0], [25, A5, 0, 0.0]]
real_strategy_VER = [[0, A3, 0, 0.0], [27, A5, 0, 0.0]]
```

---

10. Script principal `main_racesim.py`

Fichier : main_racesim.py

Options de simulation (`sim_opts`)

```
sim_opts = {
    "use_prob_infl": False,       # Active bruits gaussiens sur temps au tour, arrêts...
    "create_rand_events": False,  # Crée SC/VSC et retraits aléatoires si listes vides dans .ini
    "use_vse": False,             # Active le VSE pour les décisions de stratégie
    "no_sim_runs": 1,             # Nombre de courses à simuler (&gt;1 = Monte Carlo)
    "no_workers": 1,              # Processus parallèles (1 = séquentiel)
    "use_print": True,            # Afficher les logs
    "use_print_result": True,     # Afficher le résultat
    "use_plot": False,            # Afficher les graphiques
}
```

Flux d'exécution

```
main(sim_opts, race_pars_file, mcs_pars_file)
  ■■■ import_pars()          # Chargement .ini
  ■■■ check_pars()           # Validation
  ■■■ for each run:
        race_handle(...)     # → Race.simulate_race()
          ■■■ [postprocessing]
              ■■■ export_results_as_csv()
              ■■■ pickle du race object
```

---

11. Script principal `main_train_rl_agent_dqn.py`

Fichier : `main_train_rl_agent_dqn.py`

Hyperparamètres configurables

```
# Environnement
race = "Shanghai_2019"           # Course cible
vse_others = "reinforcement"     # VSE des autres pilotes pendant l'entraînement

# DQN
num_iterations = 250_000         # Itérations totales d'entraînement
replay_buffer_max_length = 200_000
initial_collect_steps = 200      # Steps aléatoires avant entraînement
collect_steps_per_iteration = 1
fc_layer_params = (64, 64,)      # Couches cachées du Q-network
batch_size = 64
learning_rate = 1e-3
gamma = 1.0                      # Discount factor
n_step_update = 1
target_update_period = 1
dueling_q_net = False

# Évaluation
eval_interval = 50_000
num_eval_episodes = 100
```

Architecture DQN

```
Input (observation_dim) → FC(64) → ReLU → FC(64) → ReLU → Output (nb_actions Q-values)
```

L'exploration utilise **Boltzmann** (temperature decay de 1.0 → 0.01 sur `num_iterations`).

Flux d'entraînement

[Diagramme Mermaid — voir la documentation Markdown]

Sorties de l'entraînement

Fichier	Description
---	---
`machine_learning_rl_training/output/nn_reinforcement_<race>.tflite`	Modèle TF Lite
`machine_learning_rl_training/output/preprocessor_reinforcement_<race>.pkl`	Préprocesseur

■■ **Important :** Après entraînement, **copier ces fichiers dans** `racesim/input/vse/` pour les utiliser dans `main_racesim.py`.

---

12. Modèle physique — Calcul des temps au tour

Formule générale

$$t_{lap,i} = t_q + t_{gap} + \underbrace{t_{driver,i}}_{\text{compétence pilote}} + \underbrace{t_{car,i}}_{\text{performance voiture}} + \underbrace{t_{tire}(age_{degr}, compound)}_{\text{dégradation pneus}} + \underbrace{t_{fuel,i}}_{\text{masse carburant}} + \underbrace{\epsilon}_{\text{bruit gaussien}}$$

Tour 1 uniquement :

$$t_{lap,1} = t_{lap} + t_{loss_firstlap} + (p_{grid,i} - 1) \times t_{loss_pergridpos}$$

Sous FCY (VSC) :

$$t_{lap,FCY} = \alpha \cdot t_{lap_fcy} + (1-\alpha) \cdot t_{lap,normal} \quad \text{avec } \alpha = \text{fraction du tour en FCY}$$

Avec DRS :

$$t_{lap} = t_{lap} + t_{drseffect} / 2 \quad \text{(si dans la fenêtre DRS de 1s)}$$

Arrêt au stand

$$t_{pitstop} = t_{pitdrive_inlap} + t_{pit_tirechange_min} + t_{pit_tirechange_add,team} + t_{pitdrive_outlap}$$

---

13. Référence RL complète — Espace d'états, actions, récompenses

Résumé espace d'états

#	Feature	Type	Range	Description
---	---	---	---	---
0	`cur_lap / tot_no_laps`	float	[0, 1]	Progression dans la course
1	`(position - 1) / (N - 1)`	float	[0, 1]	Position relative
2	`est_pos_loss / (N - 1)`	float	[0, 1]	Perte de positions estimée
3	`tire_age / tot_no_laps`	float	[0, 1]	Âge relatif des pneus
4+	`cur_compound`	one-hot	0/1	Composé actuel
—	`used_2compounds`	binary	0/1	2 composés utilisés
—	`fcy_stat_curlap`	categ	0-4	Statut FCY
—	`close_behind`	binary	0/1	Adversaire proche derrière
—	`close_ahead`	binary	0/1	Adversaire proche devant
—	`defendable_undercut`	binary	0/1	Sous-coupe défendable
—	`driver_initials`	one-hot	0/1	Identité du pilote

Espace d'actions

Action	Signification

| --- | --- |
`0`	Pas d'arrêt ce tour
`1`	Arrêt + composé le plus dur disponible
`2`	Arrêt + composé médium disponible
`3`	Arrêt + composé le plus tendre disponible

Reward shaping

Composante	Formule	Fréquence
Lap time	`0.1 * (t_avg_presim - t_curlap)`	Chaque tour
Position	`0.1 * Δpositions_gagnées`	Chaque tour
Finale	`position_theo - position_réelle`	Fin d'épisode
Pénalité 2 composés	`-10`	Fin d'épisode si violée

---

14. Guide pour modifier / étendre le RL

A. Modifier l'espace d'observations

Fichier à modifier : `rl_environment_single_agent.py`

Dans `__init__` : mettre à jour `_observation_spec` avec la nouvelle taille de vecteur
Dans `__calculate_observation()` : ajouter la nouvelle feature dans le vecteur `observation`
Mettre à jour le `Preprocessor` si la feature est catégorielle

■ **Conseil :** Des features intéressantes à ajouter : nombre de tours restants avant que les adversaires soient "obligés" de s'arrêter, intervalle exact (et non juste `close_behind`/`close_ahead`), données météo, ...

B. Modifier la fonction de récompense

Fichier à modifier : `rl_environment_single_agent.py`

Méthodes à modifier :

- `__calculate_reward_laptime()` → récompense sur le temps au tour
- `__calculate_reward_position()` → récompense sur les positions
- `__calculate_reward_final_position()` → récompense finale
- `_step()` → pondération et somme des récompenses

C. Changer d'algorithme RL

Fichier à modifier : `main_train_rl_agent_dqn.py`

L'environnement est compatible avec **tout algorithme** `tf-agents` ou adaptable à **Gymnasium/Stable-Baselines3** via un wrapper :

```
# Wrapper minimal pour Gymnasium/SB3
import gymnasium as gym
```

```
from gymnasium import spaces

class RaceEnvWrapper(gym.Env):
    def __init__(self, race_pars_file, mcs_pars_file, vse_type):
        self.env = RaceSimulation(race_pars_file, mcs_pars_file, vse_type)
        self.action_space = spaces.Discrete(4)
        self.observation_space = spaces.Box(low=0.0, high=1.0,
                                            shape=(obs_dim,), dtype=np.float32)

    def reset(self, **kwargs):
        time_step = self.env.reset()
        return time_step.observation.numpy(), {}

    def step(self, action):
        time_step = self.env.step(np.array(action, dtype=np.int32))
        obs = time_step.observation.numpy()
        reward = float(time_step.reward)
        done = time_step.is_last()
        return obs, reward, done, False, {}
```

D. Entraîner pour une nouvelle course

```
# 1. Vérifier qu'il existe un fichier .ini pour la course
ls racesim/input/parameters/pars_&lt;NomCourse&gt;_&lt;Annee&gt;.ini

# 2. Modifier main_train_rl_agent_dqn.py
race = "NomCourse_Annee"

# 3. Lancer l'entraînement
python main_train_rl_agent_dqn.py

# 4. Copier les sorties
cp machine_learning_rl_training/output/nn_reinforcement_&lt;race&gt;.tflite racesim/input/vse/
cp machine_learning_rl_training/output/preprocessor_reinforcement_&lt;race&gt;.pkl racesim/input/vse/
```

> ■ **Attention :** L'entraînement ne fonctionne **pas pour les courses mouillées** (composés I ou W utilisés). Vérifier d'abord avec le check dans `main_train_rl_agent_dqn.py`.

E. Modifier le modèle de physique

Pour changer le modèle de dégradation des pneus :

- **Paramètres** : `racesim/input/parameters/*.ini` → section `[TIRESET_PARS_<driver>]`
- **Code** : `helper_funcs/src/calc_tire_degradation.py`

Pour modifier la physique du dépassement :

- `race.py` → méthode `__handle_overtaking_track()`

F. Points d'extension RL recommandés

[Diagramme Mermaid — voir la documentation Markdown]

Publications de référence

Sujet	Référence
---	---

Simulation déterministe	Heilmeier et al. (2018), ITSC — DOI: 10.1109/itsc.2018.8570012
Simulation probabiliste / Monte Carlo	Heilmeier et al. (2020), Applied Sciences — DOI: 10.3390/app10124229
Virtual Strategy Engineer (VSE)	Heilmeier et al. (2020), Applied Sciences — DOI: 10.3390/app10217805

Documentation générée automatiquement par analyse du code source du projet `race-simulation-rl`.